CrisprClamp Quantifies Ordered Dual-Guide CRISPR Screens at 10-Billion-Read Scale with Deterministic, Bounded Memory

Yoon Sing Yap*

Independent researcher

*Correspondence: yoonatworkz@gmail.com

Abstract

We present CrisprClamp (CRISPR Combinatorial Library Admission and Memory-bounded Pair counter), a method for counting ordered guide pairs from paired-end sequencing reads in combinatorial CRISPR screens. CrisprClamp preserves guide position, reports zero counts for unobserved constructs, and assigns every input read pair to a defined class. On synthetic datasets with up to 50 million constructs and 1 billion reads, CrisprClamp produced exact counts and was 2.2- to 2.4-fold faster than DotMatch and 4.5- to 5.8-fold faster than MAGeCK2 in end-to-end workflow time. On a published CRISPRi genetic-interaction screen, CrisprClamp and three existing tools produced identical count matrices, and CrisprClamp completed counting in 13.4 s. At 50 million reference constructs, sampled workflow process-tree RSS was 4.3 GiB for CrisprClamp, 39–43 GiB for DotMatch, and 10.5 GiB for MAGeCK2. CrisprClamp also counted 10 billion read pairs across 1 billion distinct ordered constructs in 47.7 min of summed execution time across 20 reference shards on a single workstation. Before reading input, CrisprClamp checks modeled memory against a user-configured planning budget, and its final counts and error reports do not depend on thread scheduling.

1 Background

Combinatorial CRISPR screens express two single-guide RNAs (sgRNAs) in the same cell to perturb pairs of genes simultaneously [1,2]. By comparing combinatorial and single-gene perturbations, these screens map genetic interactions across distinct functional regimes, including synthetic lethality, functional buffering, and directional suppression, to systematically reconstruct parallel compensatory pathways, physical protein complexes, and linear signaling cascades [1, 2]. For example, dual-guide libraries have identified synthetic-lethal paralog pairs across human cancer cell lines that do not score in single-guide knockout screens [3, 4]. Accurately quantifying ordered guide pairs from paired-end sequencing reads is the critical first step in analyzing these screens, as errors in construct counts propagate directly into downstream normalization and interaction scoring.

However, these screens pose three main challenges for read counting. First, the number of possible ordered pairs grows with the square of the single-guide library size; for the human Brunello library of 76,441 guides [5], there are more than 5.8 billion possible ordered non-self pairs. Second, guide position carries biological meaning in many designs, such as orthologous dual-Cas screens and combined activation-knockout screens [6–8], so a counting method must treat (A, B) and (B, A) as distinct constructs. Third, deep screens now produce hundreds of millions to billions of paired reads, requiring the counting step to keep pace with this volume while still reporting reads that do not match any reference construct.

As combinatorial designs expand to pair dual-guide delivery with single-cell readouts [9, 10] and large guide collections [5], several tools have been developed to count dual-guide libraries, including

DotMatch [11], MAGeCK2 [12,13], and ScreenPro2 [14,15]. In our evaluation, ScreenPro2 required uncompressed FASTQ input, whereas DotMatch and MAGeCK2 read compressed input directly but required separate reference-conversion and output-sorting steps to produce a complete count table in reference order. Moreover, these workflows typically omit reads that fail quality checks or contain unexpected guide pairs rather than summarizing them by read class.

To address these limitations, we developed CrisprClamp (CRISPR Combinatorial Library Admission and Memory-bounded Pair counter), a tool written in Rust that counts ordered guide pairs directly from compressed paired-end FASTQ files. CrisprClamp preserves the position of each guide in the construct, outputs one count per construct in reference order (including zero counts for unobserved constructs), and classifies every input read pair for quality control. We evaluated CrisprClamp on synthetic datasets spanning 1 million to 50 million reference constructs, a published CRISPRi genetic-interaction screen [1], and a 20-shard workload containing 10 billion paired reads across 1 billion constructs. Our main design contributions are exact position-specific pair matching, compressed streaming with bounded work and buffer reuse, memory checks before reading input, and overflow-checked counts in reference order.

2 Results

2.1 Overview of the CrisprClamp workflow

CrisprClamp is a single compiled Rust program (Figure 1). Briefly, CrisprClamp reads paired compressed FASTQ files, extracts Guide A from read 1 and Guide B from read 2, matches each guide exactly against a position-specific guide list, and tests the ordered pair against the construct reference. Reads are processed in parallel across worker threads, and worker-local counts are merged at completion. CrisprClamp reports one count for every construct in reference order, including zeros, and assigns every input read pair to one of five classes: mapped, unmapped, invalid, ambiguous, or recombinant. Full implementation details are provided in the Materials and methods section.

2.2 CrisprClamp design keeps counts exact and resource use predictable

We designed CrisprClamp around the ordered construct reference so that counting preserves both guide identity and position (Figure 1). We match each extracted sequence against the guide list for its position and represent each ordered pair with two 32-bit guide indexes (64 bits in total). This pair identity distinguishes reciprocal orientations without storing sequence strings for every observed pair. We then test membership in two steps: a fast pre-filter rejects pairs that cannot be in the reference, and an exact lookup confirms each remaining pair. Because the pre-filter alone never accepts a construct, filter false positives cannot add counts to the output.

We stream compressed input directly and bound the number of batches held between decoding and counting. Workers return used record buffers for reuse, reducing repeated allocation while keeping the amount of in-flight work fixed. We also defer diagnostic message construction to error branches while retaining full input validation. Each worker updates a local count map; at completion, we merge these maps using overflow-checked addition so that any count overflow halts execution. We write one count per construct in reference order, including zeros, to preserve the full library denominator for downstream analysis.

Before reading input, we check modeled memory requirements against a user-configured budget. This check accounts for reference loading, counting, and output, including the bounded queues and reusable buffers. For references that exceed the chosen budget, users can prepare disjoint reference shards and supply the reads assigned to each shard, as in our 10-billion-read evaluation. This approach makes the size of each counting task explicit, though it does not automatically partition arbitrary mixed reads across shards. Table 4 summarizes these design choices; our final-build bench-

marks evaluate the complete system and do not isolate the speed contribution of each mechanism.

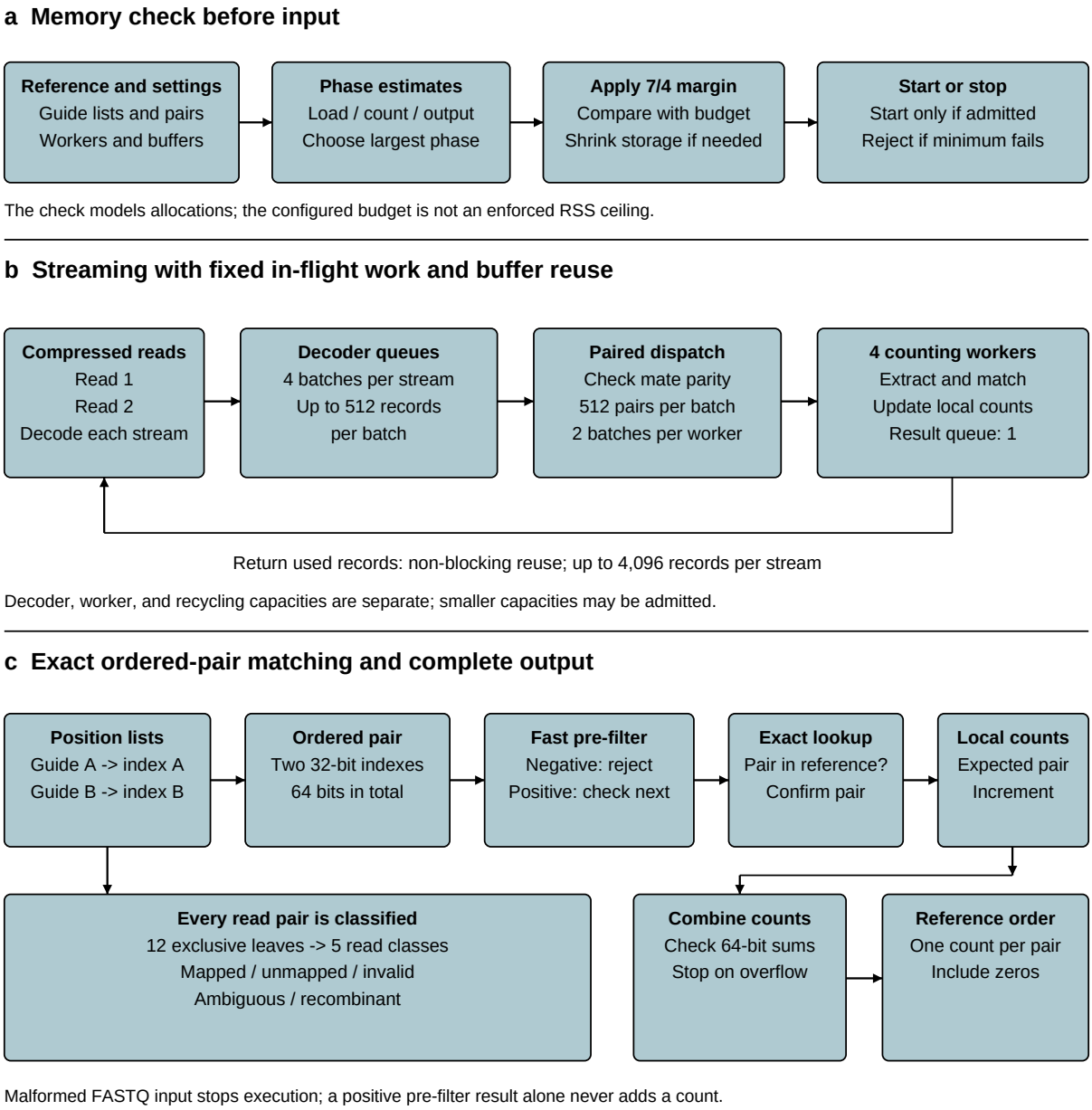


Figure 1: CrisprClamp design preserves exact counts within planned resources. (a) Before reading input, we estimate memory for reference loading, counting, and output, apply a 7/4 margin, and compare the result with the configured budget. (b) Decoder queues, counting batches, and reusable buffers bound the in-flight work. Used records return to the decoders through non-blocking recycling queues. (c) Two 32-bit guide indexes form an ordered pair identity; the pre-filter packs them into 64 bits. A fast pre-filter rejects negative candidates; every positive candidate requires exact reference membership. Twelve exclusive classification leaves form five read classes. We combine worker-local counts with overflow checks and report all constructs in reference order, including zeros.

2.3 CrisprClamp produces exact counts across synthetic libraries

Accurate construct counting is required before downstream statistical testing because counting errors distort fold-change and interaction estimates [12, 16]. To test counting accuracy against a known ground truth, we generated four synthetic paired-end datasets from the human Brunello library [5] spanning 1 million and 50 million reference constructs and 50 million to 1 billion read pairs. We then

compared CrisprClamp with DotMatch, MAGeCK2, and ScreenPro2 across these workloads. Across all 13 tool-dataset evaluations, CrisprClamp and the comparator tools produced identical counts matching the simulated ground truth with 0 discrepant constructs (Table 1). Thus, CrisprClamp recovers exact construct counts across both moderate and large reference libraries.

Table 1: Runtime scaling across synthetic datasets. Runtimes represent complete workflow durations in seconds on an Apple M4 Pro workstation (one measured run for each tool on each dataset after warmup). Stratum A contains 1,000,000 constructs in the reference; Stratum B contains 50,000,000 constructs in the reference. All 13 tool-dataset evaluations produced identical counts matching the simulated ground truth. Unreported indicates configurations not run in this cohort.

Cell	Constructs	Input pairs	Coverage	CrisprClamp	DotMatch	MAGeCK2	ScreenPro2
A-100M	1,000,000	100M	100x	32.0 s	73.4 s	145.8 s	452.7 s
A-1B	1,000,000	1B	1,000x	317.5 s	686.2 s	1,440.5 s	Unreported
B-50M	50,000,000	50M	1x	35.0 s	192.1 s	243.3 s	Unreported
B-500M	50,000,000	500M	10x	141.4 s	338.6 s	822.0 s	Unreported

2.4 CrisprClamp counts reads 2.1-fold faster than DotMatch and 4.5-fold faster than MAGeCK2 with a 1-million-construct reference

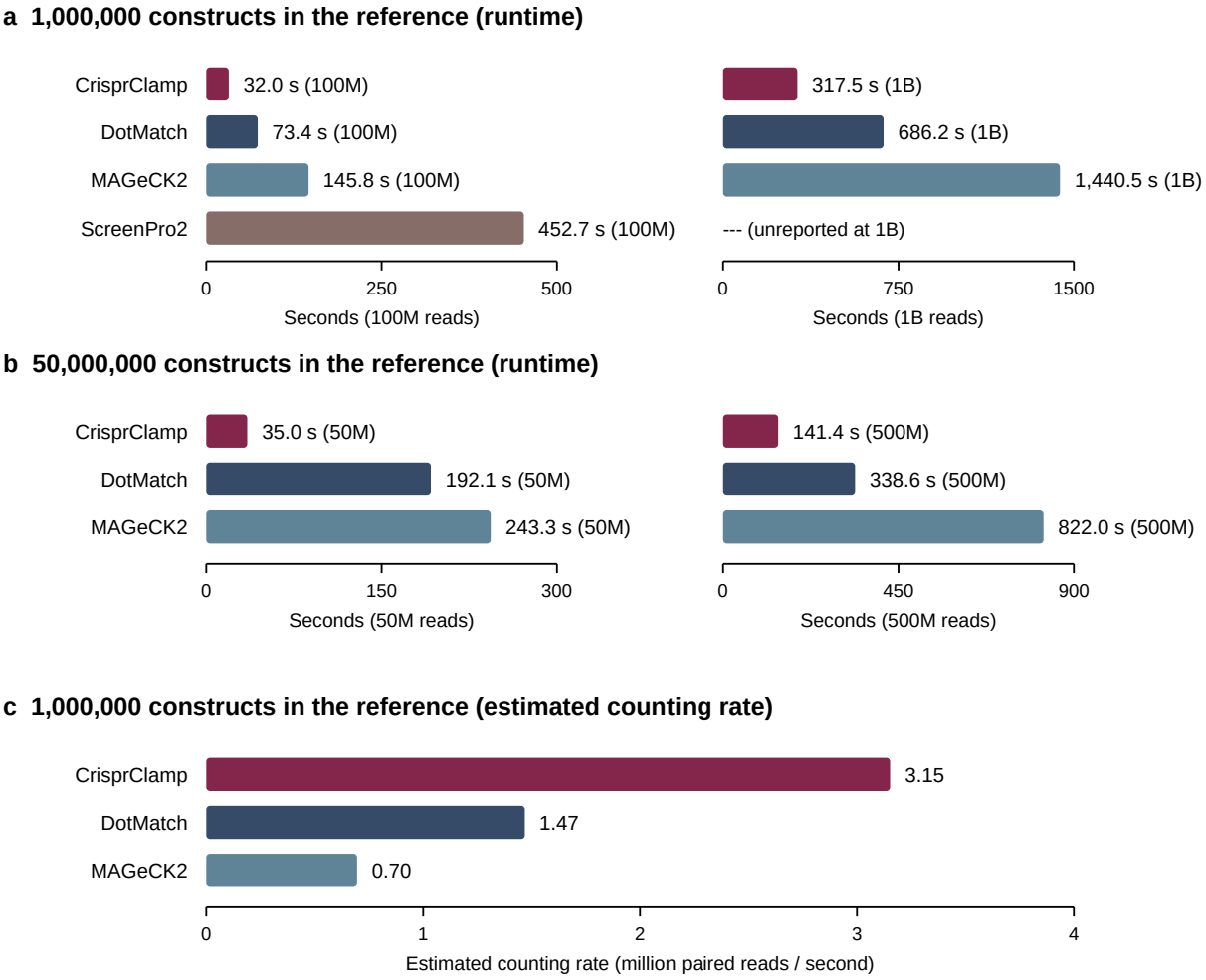
With 1 million constructs in the reference, we estimated the counting rate from the difference in read counts divided by the difference in measured workflow times (Figure 2a, c). CrisprClamp achieved an estimated counting rate of 3.15 million pairs per second, compared with 1.47 million for DotMatch and 0.70 million for MAGeCK2, corresponding to 2.1-fold and 4.5-fold higher counting rates than DotMatch and MAGeCK2, respectively. These values are two-run workflow estimates rather than isolated measurements of the counting stage; ScreenPro2 was evaluated at a single depth and was therefore excluded from this calculation.

2.5 CrisprClamp is the fastest tool with a 50-million-construct reference

With 50 million constructs in the reference, CrisprClamp completed the 50-million-read and 500-million-read workflows in 35.0 s and 141.4 s, respectively, compared with 192.1 s and 338.6 s for DotMatch and 243.3 s and 822.0 s for MAGeCK2 (Figure 2b). Because the two 50-million-construct inputs differ in compression ratio, we do not estimate a two-point counting rate or fixed cost for this reference.

2.6 CrisprClamp produces counts identical to existing tools on a published screen

Because synthetic reads do not contain sequencing errors or unexpected sequences, we next applied CrisprClamp, DotMatch, MAGeCK2, and ScreenPro2 to sample SRR7470444 from a published CRISPRi genetic-interaction screen [1]. This sample contains 41.9 million paired reads against a reference of 1,044,484 ordered pairs. All four tools produced identical count matrices with 29.6 million mapped read pairs (Table 2, Figure 3a). CrisprClamp completed counting in 13.4 s, compared with 15.4 s for DotMatch, 72.4 s for MAGeCK2, and 162.0 s for ScreenPro2. CrisprClamp also classified every input read pair: 70.7% were mapped, 27.2% were unmapped, and 2.1% were invalid (Figure 3b). The recombinant class was empty by construction on this sample, because the reference contains every ordered pair of the 1,022 guides, including self-pairs; the ambiguous class was also empty. Simple workflow throughput was 3.13, 2.72, 0.58, and 0.26 million pairs per second for CrisprClamp, DotMatch, MAGeCK2, and ScreenPro2, respectively (Figure 3c). Because we measured one run per tool, fixed startup costs cannot be separated from per-read costs. This read-class summary allows users to inspect library representation and read quality before downstream statistical analysis.

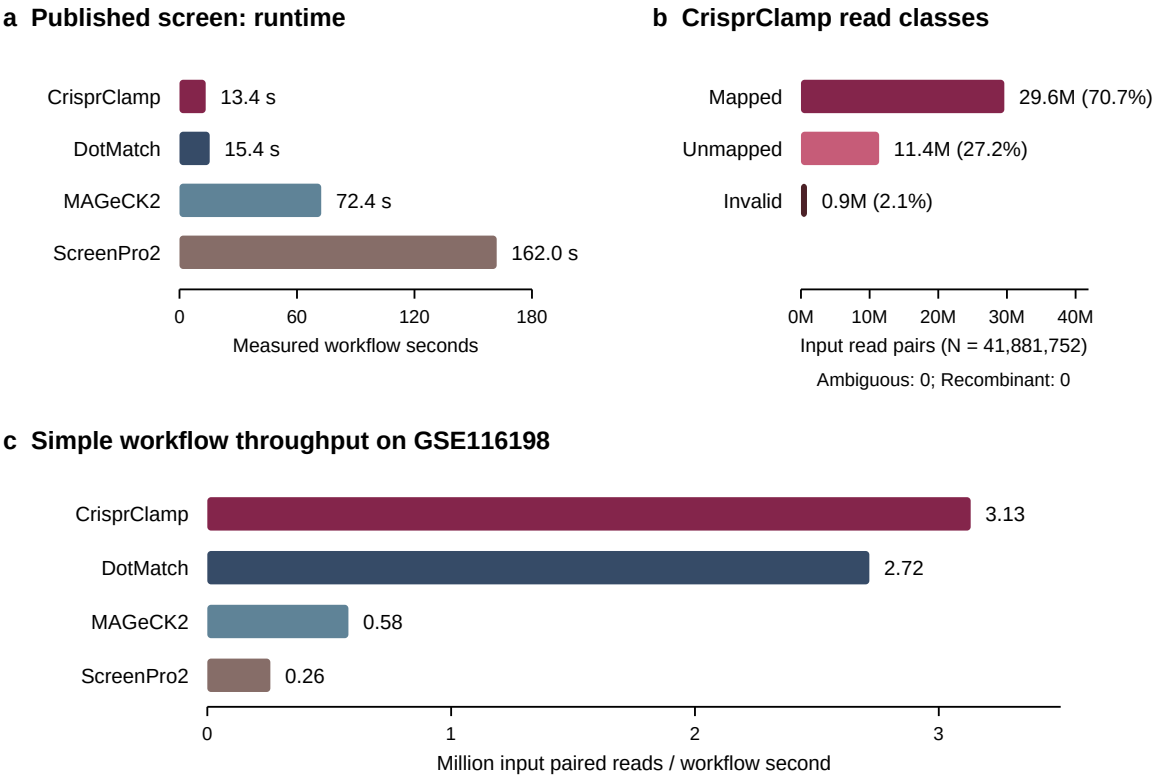


Two-run estimates; no error bars.

Figure 2: Measured runtimes and estimated counting rates. (a, b) Measured workflow runtimes for the 1-million-construct and 50-million-construct references. (c) Estimated counting rate, $(N_2 - N_1)/(T_2 - T_1)$, in millions of pairs per second, from two measured depths with the 1-million-construct reference: 3.15 for CrisprClamp, 1.47 for DotMatch, and 0.70 for MAGeCK2. ScreenPro2 was measured at one depth only and is not shown. These are two-run workflow estimates, not isolated measurements of counting stages. The two inputs for the 50-million-construct reference differ in compression, so no counting rate or fixed cost is estimated for that reference. No error bars are shown. Each quantitative panel has a CSV available from the author on request with measured inputs and the applicable formula.

Table 2: Validation on a published CRISPRi genetic-interaction screen (GSE116198). Evaluated on sample SRR7470444 (41,881,752 paired-end reads) against a 1,022-guide reference library containing 1,044,484 ordered pairs. Runtimes represent complete workflows on an Apple M4 Pro workstation. All four tools produced identical count matrices with 29,620,327 mapped read pairs.

Tool	Constructs	Input pairs	Runtime	Mapped count	Matrix agreement
CrisprClamp	1,044,484	41,881,752	13.4 s	29,620,327	Identical (100%)
DotMatch	1,044,484	41,881,752	15.4 s	29,620,327	Identical (100%)
MAGeCK2	1,044,484	41,881,752	72.4 s	29,620,327	Identical (100%)
ScreenPro2	1,044,484	41,881,752	162.0 s	29,620,327	Identical (100%)



One measured run per tool; fixed cost cannot be separated.

Figure 3: Validation on a published CRISPRi genetic-interaction screen (GSE116198). (a) Workflow runtime (seconds) on sample SRR7470444 (41,881,752 paired reads; 1,044,484 constructs in the reference). CrisprClamp completed counting in 13.4 s, compared with 15.4 s for DotMatch, 72.4 s for MAGeCK2, and 162.0 s for ScreenPro2. All four tools produced identical count matrices with 29,620,327 mapped read pairs. (b) CrisprClamp assignment of all 41,881,752 input read pairs into five read classes: 29,620,327 mapped (70.7%), 11,385,600 unmapped (27.2%), 875,825 invalid (2.1%), 0 ambiguous, and 0 recombinant. Panels a and b are each backed by a CSV file available from the author on request. (c) Simple throughput is input paired reads divided by measured workflow seconds, in millions of pairs per second. One run per tool is available, so fixed cost cannot be separated. The additional panel has a CSV available from the author on request with the formula.

2.7 CrisprClamp completes a 10-billion-read workload on a single workstation

Large combinatorial libraries sequenced at high coverage can produce billions of reads. To test CrisprClamp at this scale, we generated 1 billion distinct ordered constructs at 10-fold coverage (10 billion paired reads in total), partitioned into 20 shards of 500 million reads each. CrisprClamp completed all 20 shards in 47.7 min of summed execution time, whereas DotMatch completed them in 110.7 min of summed workflow time (Figure 4a, Table 3). For MAGeCK2, we measured three 500-million-read shards and projected the 20-shard workload from the median shard time, yielding a projected total of 274.6 min. Across all three tools, per-shard runtimes were consistent with their runtimes on the 500-million-read synthetic dataset and remained within approximately 4% of their respective medians, supporting the MAGeCK2 projection (Figure 4b). For both CrisprClamp and DotMatch, shard 0 was the slowest shard. Because the DotMatch and MAGeCK2 totals include format-conversion and sorting steps that are not part of the CrisprClamp execution total (see Materials and methods), we do not report a direct speedup ratio for this workload. Dividing 500 million pairs by the mean measured shard time yielded 3.50 million pairs per second for CrisprClamp, 1.51 million for DotMatch, and 0.61 million for MAGeCK2 (based on three measured shards; Figure 4c). These derived throughputs retain the distinct timing boundaries of each workflow.

Table 3: 10-billion-read workload summary and timing boundaries. Performance across 20 shards containing 1 billion distinct ordered constructs in the reference and 10 billion paired-end reads on an Apple M4 Pro workstation. The CrisprClamp total is summed execution time from process launch to completion detection; the DotMatch total is summed workflow time including format conversion and sorting; the MAGeCK2 total is projected from the median of three measured 500-million-read shards, with a descriptive range of 272.3–278.8 min.

Tool	Measured shards	Target shards	Total duration	Status and timing boundary
CrisprClamp	20 of 20	20	47.7 min (2,859.8 s)	Measured execution sum
DotMatch	20 of 20	20	110.7 min (6,641.9 s)	Measured workflow sum
MAGeCK2	3 of 20	20	274.6 min (16,476.9 s)	Projected workflow total

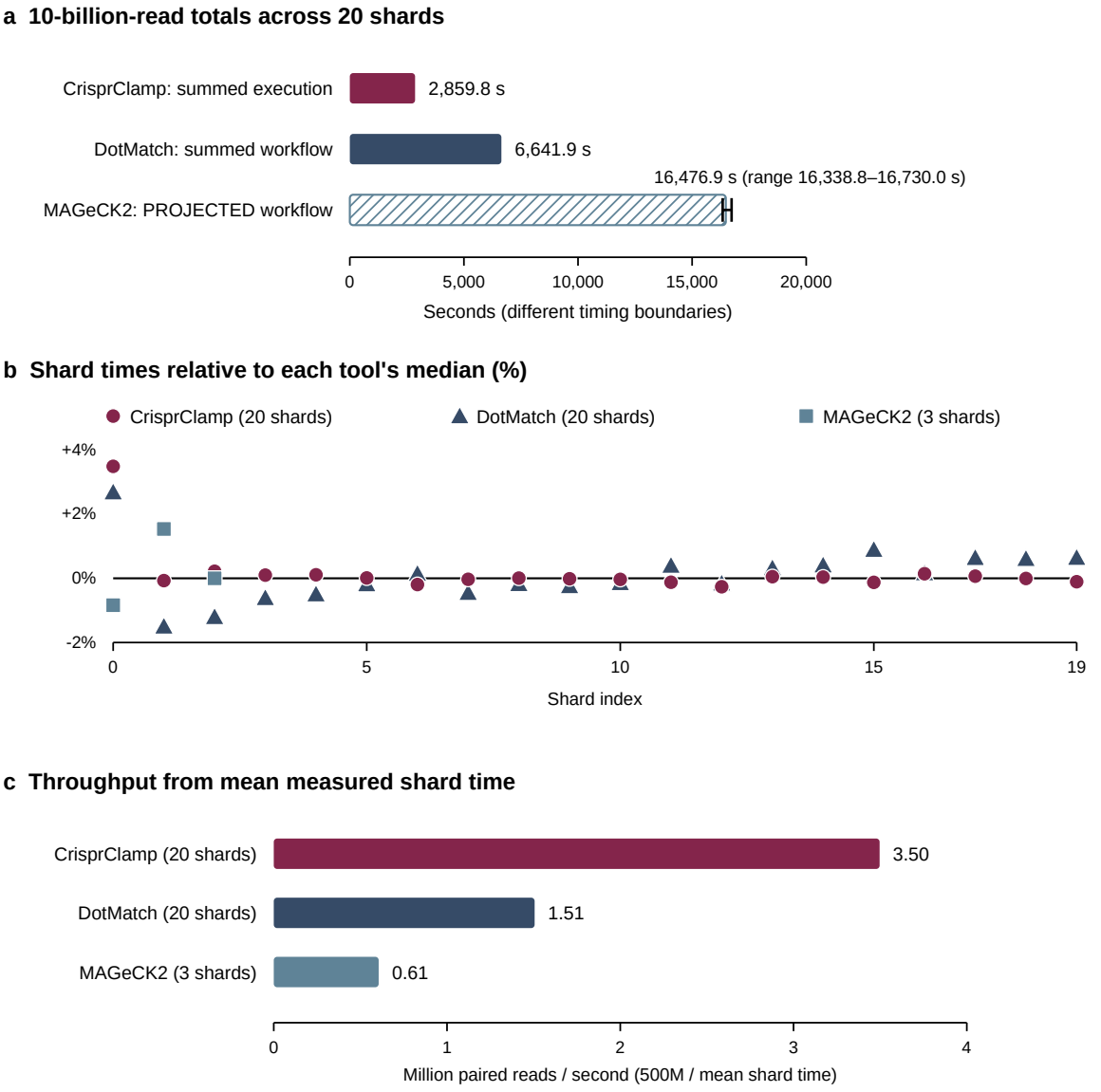


Figure 4: 10-billion-read workload performance across 20 shards. (a) Total execution time across 20 shards (1 billion distinct ordered constructs in the reference and 10 billion paired reads). CrisprClamp completed all 20 shards in 2,859.8 s (47.7 min) of summed execution time, and DotMatch completed all 20 shards in 6,641.9 s (110.7 min) of summed workflow time. The projected MAGeCK2 10-billion-read runtime from three measured 500-million-read shards is 16,476.9 s (274.6 min), shown as a hatched bar with whiskers indicating the descriptive range (16,338.8–16,730.0 s) obtained by scaling the minimum and maximum measured shard times by 20. This descriptive projection range is not a confidence interval or standard error. Because the totals use different timing boundaries, we do not report a direct speedup. (b) Measured shard times as percentage differences from each tool's own median, $(t - median)/median \times 100$: 20 CrisprClamp shards, 20 DotMatch shards, and three MAGeCK2 shards. Each marker represents one measured shard (CrisprClamp circles, DotMatch triangles, and MAGeCK2 squares); the black line at 0% marks the median. Median shard times are 142.8 s for CrisprClamp, 331.8 s for DotMatch, and 823.8 s for MAGeCK2. Full per-shard times are in Supplementary Table S3. Each shard contains 50,000,000 reference constructs and 500,000,000 paired reads at 10-fold coverage. (c) Throughput from mean measured shard time: 500 million pairs divided by mean measured shard seconds, giving 3.50, 1.51, and 0.61 million pairs per second for CrisprClamp, DotMatch, and MAGeCK2. These derived simple rates retain the different timing boundaries and do not provide a direct speedup. Each quantitative panel has a CSV available from the author on request with its inputs and formula.

2.8 CrisprClamp memory depends on reference size, not read depth

To separate the memory required by the construct reference from that required by sequencing depth, we examined the maximum sampled process-tree resident set size (RSS) recorded during the final-build workflows. With 1 million reference constructs, CrisprClamp used 425.6 MiB at 100 million paired reads and 430.0 MiB at 1 billion paired reads. With 50 million reference constructs, it used 4.31 GiB at both 50 million and 500 million paired reads (Figure 5a). Thus, across the tested read depths, memory remained nearly constant for a fixed reference and scaled primarily with reference size. These values are single-run workflow measurements sampled once per second rather than kernel-recorded native-process maxima, so brief allocation peaks between samples may not be captured.

We compared memory across tools only where benchmark records shared identical inputs, host hardware, and sampling methods. On the 50-million-construct datasets, maximum sampled workflow process-tree RSS was 4.31 GiB for CrisprClamp 0.1.0, 39.0–43.1 GiB for DotMatch 0.6.3, and 10.5 GiB for MAGeCK2 0.3.0 (Figure 5b). On the published screen (GSE116198), the corresponding measurements were 195.2 MiB for CrisprClamp 0.1.0, 119.4 MiB for DotMatch 0.6.3, 2,748.3 MiB for MAGeCK2 0.3.0, and 8,869.0 MiB for ScreenPro2 0.7.0 (Figure 5c). Because these measurements include reference preparation and output conversion processes, they reflect the complete evaluated workflows rather than an isolated counting stage. At smaller reference sizes, both CrisprClamp (430.0 MiB) and MAGeCK2 (324.1 MiB) used less than 0.5 GiB on the 1-million-construct, 1-billion-read dataset, and DotMatch used less memory than CrisprClamp on GSE116198 (119.4 MiB vs. 195.2 MiB); the largest memory differences emerged with the 50-million-construct reference (Figure 5d). Comparing GSE116198 (1,044,484 constructs) with the 50-million-construct, 500-million-read synthetic dataset yielded estimated memory increments of 90.4 bytes per additional construct for CrisprClamp, 852.0 bytes for DotMatch, and 172.1 bytes for MAGeCK2 (Figure 5e). Because the two endpoints compare an experimental screen (41.9 million paired reads, 19-nt guides) with a synthetic dataset (500 million paired reads, 20-nt guides), this slope includes workload, preparation, and conversion differences rather than an isolated per-construct cost.

We configured memory budgets of 2.00 GiB for the smaller references and 24.0 GiB for the 50-million-construct references. The current exact-membership loader admits at most 50 million constructs per counting task; larger references require disjoint shards. At startup, the admission check multiplies the largest modeled phase requirement by 7/4 and rejects a task if its minimum admitted geometry exceeds the budget, accounting for fixed queue and buffer capacities (Materials and methods). The configured budget serves as an admission planning limit rather than an OS-enforced RSS ceiling, and actual resident memory can vary with allocator behavior, reference occupancy, record length, and worker geometry.

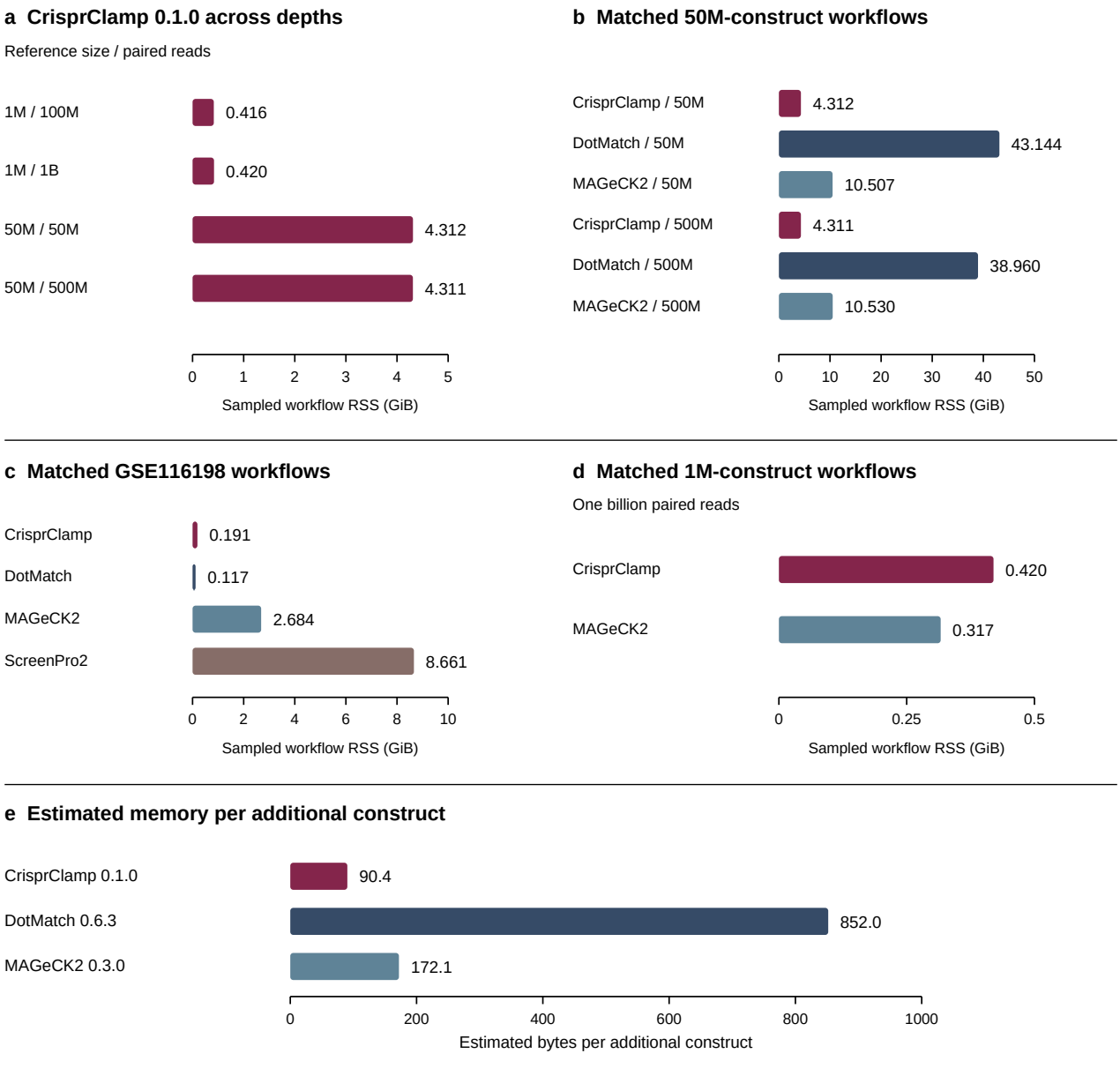


Figure 5: Measured workflow memory and estimated memory per additional construct. (a) CrisprClamp 0.1.0 maximum sampled workflow process-tree RSS at two depths per reference. (b) Matched 50-million-construct workflows for CrisprClamp 0.1.0, DotMatch 0.6.3, and MAGeCK2 0.3.0. (c) Matched GSE116198 workflows, including ScreenPro2 0.7.0. (d) Matched 1-million-construct, 1-billion-read CrisprClamp and MAGeCK2 workflows. (e) Estimated bytes per additional construct, $(RSS_2 - RSS_1)/(M_2 - M_1)$, using GSE116198 (1,044,484 constructs) and the synthetic dataset with 50,000,000 constructs for all three tools. The two points are a real screen with 41.9 million paired reads and 19-nt guides and a synthetic dataset with 500 million paired reads and 20-nt guides, so the slope includes workload differences, preparation, and conversion and is not an isolated per-construct cost. Panels a–d report GiB; one GiB is 1,073,741,824 bytes. Each measured observation is one run sampled approximately once per second; brief peaks may be missed. These values are not native-process kernel maxima. Each quantitative panel has a CSV available from the author on request with exact inputs and formulas. No error bars are shown.

Table 4: CrisprClamp design choices and their functions. Values describe the final evaluated build; the admission planner can reduce physical buffer and queue capacities. The table does not assign runtime gains to individual mechanisms.

Mechanism	Function	Why it matters
Position-specific pair identity	Two 32-bit guide indexes; 64 bits in total	Preserves reciprocal guide orientations
Two-step membership	Fast pre-filter followed by exact reference lookup	Filter positives alone cannot add counts
Compressed streaming	Decode each read stream directly; bounded decoder and worker queues	Keeps in-flight work independent of total reads
Batch dispatch	Four workers; 512 pairs per batch; two queued batches per worker	Makes worker storage explicit in the memory model
Buffer reuse	Non-blocking return of up to 4,096 records per stream	Reuses allocated record storage without blocking workers
Deferred diagnostics	Construct error messages on error branches	Retains validation while reducing routine allocation
Local counts and checked merge	Expected-pair sparse maps; checked unsigned 64-bit sums	Avoids locks during updates and stops on overflow
Complete output	One unsigned 64-bit count per reference construct, including zeros	Retains construct order and the full denominator
Memory admission and sharding	Phase maximum with 7/4 margin; at most 50 million constructs per task	Checks the configured budget before input and bounds shard size

3 Discussion

We developed CrisprClamp to count ordered guide pairs from paired-end reads in combinatorial CRISPR screens. In screens that use orthologous Cas enzymes or combine gene activation with gene knockout [6–8], the two positions in a construct have distinct biological roles. By keeping (Guide A, Guide B) separate from (Guide B, Guide A) and reporting one count per construct in reference order, CrisprClamp preserves directional information while allowing downstream tools to pool reciprocal orientations when desired. Reporting zero counts for unobserved constructs also preserves the full reference denominator needed to detect construct dropout during negative selection.

Our benchmarks evaluate sequencing depth and reference library size across the tested workflows, although differences in input compressibility limit direct rate comparisons across reference sizes. For libraries reaching hundreds of millions to 1 billion constructs in the reference, partitioning the reference into disjoint shards allows CrisprClamp to count billions of reads on a single workstation.

With the 1-million-construct reference held constant, increasing read depth 10-fold scarcely changed CrisprClamp memory usage. The number of reference constructs, worker settings, and bounded buffers therefore provide a basis for predicting memory requirements before a run, while the 50-million-construct results show why reference size must remain part of that estimate. Users can check the modeled requirement before counting and select an admissible shard size. Nevertheless, the configured budget remains a planning limit rather than an enforced RSS ceiling, as allocator behavior and workflow preparation can affect measured memory.

CrisprClamp focuses on read counting and quality control rather than statistical modeling. In a typical workflow, CrisprClamp converts paired compressed FASTQ files into count matrices and read-class summaries (mapped, unmapped, invalid, ambiguous, and recombinant), which help users identify cloning dropouts, low read quality, or unexpected guide combinations early. Downstream packages such as MAGeCK, MAGeCKFlute, and ScreenPro2 [12, 14–16] then use these counts to normalize samples, estimate fold changes, and score genetic interactions.

This architecture reflects a broader counting pattern: extract sequences at fixed positions, convert them to integer indexes, combine those indexes into a fixed-width key, and output counts in reference order. Future work could append a unique molecular identifier (UMI) to the construct key to

distinguish individual molecules, following the molecular counting principle of Kivioja and colleagues [17]. Because counting distinct UMIs requires maintaining a set per construct rather than a single scalar counter, memory would grow with the number of observed molecules, and UMI sequencing errors would require error-tolerant matching. Such an extension would alter both the memory model and the validation rules.

For single-cell barcode tracking, a future extension could replace Guide A with a cell barcode and use a barcode whitelist as its position-specific dictionary, using sparse output to avoid writing every possible cell-by-guide combination. Single-cell perturbation screens [9, 10] and the clonal tracking strategy of CRISPR-UMI [18] motivate these extensions, although cellular lineage labels and molecule identifiers require distinct counting rules. Indexing three or more guides, or lineage labels that exceed the current two-index representation, would require keys wider than 64 bits. These extensions remain proposed adaptations; we have not implemented or evaluated these counting modes in CrisprClamp.

This study has four main limitations. First, each synthetic and experimental benchmark represents a single measured run after warmup on an Apple M4 Pro workstation rather than replicated measurements across multiple hardware architectures. In addition, the synthetic inputs use constant quality scores and gzip level 1, so their compression ratios differ from those of experimental sequencing data. Second, we evaluated exact sequence matching within fixed read windows; screens with high sequencing error rates or variable-length indels may require approximate matching. Third, the exact-membership loader accepts at most 50 million constructs per counting task, so larger references require disjoint shards. Our 10-billion-read evaluation partitioned the 1-billion-construct reference into 20 shards of 50 million constructs each, showing that sharded counting is practical on a workstation without testing a single unsharded 1-billion-construct run. Fourth, the MAGeCK2 10-billion-read total is projected from three measured shards, and the 10-billion-read totals use different timing boundaries across tools, so we do not claim a direct speedup for that workload.

4 Conclusions

CrisprClamp is a fast, exact method for counting ordered guide pairs in combinatorial CRISPR screens. It produces counts identical to existing tools on synthetic and published experimental datasets, scales to references with 50 million constructs and sharded workloads of 10 billion paired reads, classifies every input read pair for quality control, and uses 4.3 GiB of sampled workflow memory at 50 million constructs. CrisprClamp can be integrated directly into existing CRISPR screen analysis pipelines to accelerate read counting on a standard workstation.

5 Materials and methods

5.1 Ordered dual-guide data model and reference compilation

Let G_A and G_B denote the single-guide sequence lists for construct positions A and B. When both positions draw from a shared list of n guides and self-targeting pairs are excluded, the number of possible ordered non-self pairs is:

$$U = n(n-1) \tag{1}$$

Because position A and position B can use different promoters or Cas enzymes, CrisprClamp treats ordered pairs as directionally distinct:

$$(g_A, g_B) \neq (g_B, g_A) \tag{2}$$

To represent constructs compactly, we map each unique guide sequence in G_A and G_B to a 32-bit unsigned integer index (g_A and g_B) and retain each ordered pair as a two-index key with a 64-bit payload. For the Bloom pre-filter, we pack these indexes into a 64-bit unsigned integer:

$$k = (g_A \ll 32) \mid g_B \tag{3}$$

Constructs in the reference are compiled into a little-endian binary table accompanied by a descriptor that records the total pair count, starting ordinal, and self-pair policy. At startup, CrisprClamp loads exact pair membership into a sparse table preceded by a Bloom pre-filter [19, 20]. The filter allocates at least 15 bits per reference pair, rounds its storage to a power of two with a 256-byte minimum, and uses 10 probes. This configuration yields a modeled false-positive probability below 0.1%, with the 1-million-pair and 50-million-pair filters occupying 2 MiB and 128 MiB, respectively. Every positive filter result still requires an exact membership lookup.

5.2 Parallel streaming, read classification, and output generation

CrisprClamp processes paired-end reads using a producer-worker-consumer architecture connected by bounded channels. A decoder thread for each compressed read stream checks FASTQ framing and supplies batches of at most 512 records through a forward queue of four batches per stream [21]. The coordinator verifies mate parity and dispatches paired records to four counting workers, with two batches of 512 read pairs queued per worker and a result queue capacity of one. The 4,096 paired records in the worker queues represent only part of the in-flight state: each decoder also maintains producer-owned and consumer-pending batches, along with a recycling queue holding up to 4,096 consumed records per stream. Workers return consumed records through non-blocking queues so that a full recycling queue cannot stall counting. Under a tighter memory budget, the admission planner can reduce physical batch sizes, worker queue capacity, and recycling capacity while preserving the logical partition of reads. We construct diagnostic messages only on error branches while retaining structural validation of every record.

For each read pair, workers extract Guide A from read 1 and Guide B from read 2 using configured coordinate windows (20 nt for standard Cas9 libraries). If both sequences match the position-specific guide dictionaries, we test the packed key against the Bloom pre-filter and confirm the ordered two-index key against the exact construct table. Every input read pair is assigned to one of 12 mutually exclusive classification leaves that aggregate into five read classes (mapped, unmapped, invalid, ambiguous, and recombinant; Supplementary Table S1), conserving all input read pairs:

$$N_{\text{input}} = \sum_{\ell=1}^{12} N_\ell \tag{4}$$

Each worker accumulates counts in a local sparse map containing only observed expected pairs, avoiding locks on the per-read update path. When the input stream ends, worker count maps are merged via overflow-checked reduction using 64-bit unsigned integer addition. CrisprClamp writes one count per construct in reference order as a headerless binary array of 64-bit little-endian unsigned integers:

$$B_{\text{counts}} = 8M \text{ bytes} \tag{5}$$

Unobserved constructs in the reference receive explicit zero counts. Output files are written to a staging directory before atomic publication alongside JSON completion metadata. Before allocating worker structures, a format-aware admission planner calculates modeled peak memory across the loading, counting, and output phases with a 1.75 (7/4) safety factor (Supplementary Equation S3) and halts execution if the required memory exceeds the configured budget.

5.3 Computational evaluation

All benchmarks were run on an Apple M4 Pro workstation with 14 CPU cores (10 performance cores and 4 efficiency cores), 48 GiB of unified memory, and macOS 15.8 (Darwin 24.6.0, arm64). Each synthetic and experimental benchmark was run once using monotonic system timers after an operational warmup pass. CrisprClamp was compiled with Rust 1.98.0 (aarch64-apple-darwin) in release mode with optimization level 3 and Thin LTO. We compared CrisprClamp 0.1.0 with DotMatch 0.6.3 [11], MAGeCK2 0.3.0 [12,13], and ScreenPro2 0.7.0 [14,15].

We ran CrisprClamp 0.1.0 in exact dual-guide mode (`--mode dual --workers 4 --batch-records 512 --work-queue-capacity 2 --result-queue-capacity 1 --radius 0 --no-gpu`) with a memory budget of 2.00 GiB for the 1-million-construct synthetic and GSE116198 datasets and 24.0 GiB for the 50-million-construct synthetic and 10-billion-read datasets. We selected four workers and 512-record batches based on development tests (Supplementary Section S8); this configuration was the fastest among the tested settings during development, rather than a universal optimum across hosts. CrisprClamp reads compressed FASTQ input directly and writes binary counts in reference order without intermediate text conversion or external sorting. For the synthetic and GSE116198 datasets, the timed boundary covered binary reference preparation and read counting through the final output flush. For the 10-billion-read workload, CrisprClamp was timed from process launch to completion detection across all 20 shards (including ≤ 5 s per shard of completion polling and exit overhead, and excluding external reference preparation and archiving).

We ran DotMatch 0.6.3 in exact paired-guide counting mode (`dotmatch pair-count --k 0 --metric hamming`). DotMatch reads compressed FASTQ files directly and outputs observed target pairs in text format. Because DotMatch omits unobserved constructs and does not output counts in reference order, its timed boundary included converting the reference to a target list, running `dotmatch pair-count`, mapping pairs to reference order, external numerical sorting, and filling zero counts for unobserved constructs.

We ran MAGeCK2 0.3.0 in paired-guide counting mode (`mageck2 count --pairguide secondpair --norm-method none --unmapped-to-file`), which reads compressed FASTQ files directly. Because MAGeCK2 requires a text guide library and outputs a text count table, its timed boundary included reference format conversion, running `mageck2 count`, mapping pairs to reference order, external numerical sorting, and filling zero counts for unobserved constructs.

We ran ScreenPro2 0.7.0 using its dual-guide exact counting workflow with eight Polars threads (`POLARS_MAX_THREADS=8`). Because ScreenPro2 requires uncompressed FASTQ files with space-delimited read identifiers, its timed boundary included decompressing and normalizing the input FASTQ files, running ScreenPro2 counting, and writing the count matrix in reference order.

For each tool with two measured read depths on the 1-million-construct reference, we estimated the counting rate from the two observations:

$$R_{\text{estimated}} = \frac{N_2 - N_1}{T_2 - T_1} \tag{6}$$

where N_1 and N_2 are input paired-read counts, T_1 and T_2 are measured workflow seconds, and $R_{estimated}$ is pairs per second. We label this quantity estimated and report no error bars. A single run, including each tool on GSE116198, cannot identify this rate, and two observations cannot test linearity. We do not apply this two-point estimate to the 50-million-construct reference because its two inputs differ in compression.

We calculated simple throughput directly from each timed workload:

$$Q_{\text{simple}} = N/T \tag{7}$$

For Figure 4c, N is 500 million paired reads and T is the mean measured shard time across 20 shards for CrisprClamp, 20 shards for DotMatch, and three shards for MAGeCK2. This quantity is

the ratio using mean shard time rather than the arithmetic mean of per-shard rates. Because the CrisprClamp boundary spans process launch to completion detection whereas comparator boundaries include preparation and output ordering, we do not infer a direct speedup from these rates.

For Figure 5e, we estimated memory per additional construct from the difference in sampled workflow process-tree RSS divided by the difference in reference sizes:

$$b_{\text{estimated}} = \frac{\text{RSS}_2 - \text{RSS}_1}{M_2 - M_1} \tag{8}$$

For all three tools, observation 1 is GSE116198 (1,044,484 constructs, 41.9 million paired reads, 19-nt guides) and observation 2 is the 50-million-construct, 500-million-read synthetic dataset (20-nt guides). Here, RSS is in exact bytes and M is the number of reference constructs. Because the two endpoints differ in read depth, guide length, and preparation/conversion steps, this slope does not represent an isolated per-construct memory cost.

5.4 Synthetic, experimental, and 10-billion-read datasets

We generated four synthetic paired-end datasets from the 76,441-guide human Brunello library [5]. Two datasets used 1,000,000 reference constructs at 100 million paired reads (100-fold coverage) and 1 billion paired reads (1,000-fold coverage). Two datasets used 50,000,000 reference constructs at 50 million paired reads (1-fold coverage) and 500 million paired reads (10-fold coverage). Guides were placed at forward coordinates $[30, 50)$ on read 1 and read 2 with zero simulated sequencing errors or recombinants. Although all synthetic mates were 100 nt, compressed bytes per paired read differed across datasets (28.9 bytes for the 1-million-construct inputs vs. 11.6 bytes for the 500-million-read, 50-million-construct input). Specifically, the 500-million-read input places 10 copies of each construct consecutively, enabling stronger gzip compression than the single-copy 50-million-read input or the 1-million-construct inputs, which cycle through the reference; all inputs use gzip level 1 and constant quality strings.

For experimental data validation, we analyzed sample SRR7470444 (GEO accession GSM3213840; 41,881,752 paired-end reads) from a published CRISPRi genetic-interaction screen (GSE116198) [1]. We extracted 19-nt guide sequences from forward positions $[0, 19)$ on read 1 and read 2 and matched them against the 1,022-guide reference library comprising all 1,044,484 ordered pairs (including self-pairs). We counted ordered guide pairs from this sample directly and did not reproduce the third-read guide-barcode concordance workflow of the original study.

For the 10-billion-read workload, we partitioned 1 billion distinct ordered constructs into 20 disjoint shards of 50,000,000 constructs each. At 10-fold coverage, each shard contained 500,000,000 paired reads (10,000,000,000 paired reads in total), with 10 copies of each construct placed consecutively as in the 500-million-read synthetic input. CrisprClamp and DotMatch were measured across all 20 shards (Figure 4b, Supplementary Table S3). MAGeCK2 was measured on shards 0, 1, and 2 (150 million constructs and 1.5 billion paired reads; Figure 4b, Supplementary Table S3), completing in 816.9 s, 836.5 s, and 823.8 s. We projected the 20-shard MAGeCK2 workflow time by multiplying the median shard time (823.8 s) by 20:

$$T_{20}^{\text{projected}} = 20\,\text{median}(t_0, t_1, t_2) = 16{,}476.9 \text{ s } (\approx 274.6 \text{ min}) \tag{9}$$

We also scaled the minimum (816.9 s) and maximum (836.5 s) measured shard times by 20 to obtain a descriptive range of 16,338.8 s (272.3 min) to 16,730.0 s (278.8 min).

Throughput was calculated as input paired reads divided by workflow seconds, and speedup ratios were calculated as comparator runtime divided by CrisprClamp runtime on the same dataset. Main-text runtimes and ratios are rounded to one decimal place, while full numeric precision is preserved in CSV files available from the author on request.

6 Funding

This work received no specific funding.

7 Competing interests

The author declares no competing interests.

8 Acknowledgments

None.

9 Data and code availability

Experimental dual-guide sequencing data from the published CRISPRi genetic-interaction screen are publicly available from the NCBI Sequence Read Archive under accession SRR7470444 (BioProject PRJNA477770, GEO accession GSE116198) [1]. CrisprClamp 0.1.0 source code is available at https://github.com/Yoonecorn/crisprclamp under the MIT OR Apache-2.0 open-source license. Benchmark analysis scripts, figure rendering code, and quantitative panel CSV files are available from the author on request.

Supplementary Methods

S1 Reference semantics, self-pair policy, and ordinal mapping

CrisprClamp maintains two complementary concepts of construct identity: the ordered guide pair defines the biological construct, whereas its ordinal defines its fixed position in the construct reference. Permuting reference rows changes ordinal assignments even when the set of guide pairs is unchanged. Downstream tools require both the single-guide dictionaries and the ordered reference table to interpret count vectors.

The versioned reference descriptor specifies the pair-stream path, total pair count, initial ordinal, and self-pair policy. Each reference entry stores two unsigned 32-bit little-endian indices representing Guide A and Guide B. Descriptor loading validates version compatibility, expected dimensions, and stream length before counting begins.

When self-targeting pairs are excluded by library policy, equality is tested on the nucleotide sequence across positions rather than on index values. In the GSE116198 dataset, self-targeting pairs are retained by the library design, giving $1{,}022^2 = 1{,}044{,}484$ ordered pairs in the reference.

S2 Read classification and input conservation

The 12 classification leaves of CrisprClamp's accounting model cover expected mapping; unexpected recombinant mapping; component and layout ambiguity; layout unmapped; Position A-known, Position B-known, and neither-known unmapped; and alphabet, extraction, ambiguous-base, and quality invalidity (Supplementary Table S1). Precedence rules ensure that each input read pair increments exactly one classification leaf.

Let c_i denote the count assigned to construct i in the reference, and let N_{mapped} denote total mapped read pairs. The mapped matrix sum and total input conservation are given by:

$$N_{\text{mapped}} = \sum_{i=1}^{M} c_i \tag{S1}$$

$$N_{\text{input}} = N_{\text{mapped}} + N_{\text{unmapped}} + N_{\text{invalid}} + N_{\text{ambiguous}} + N_{\text{recombinant}} \tag{S2}$$

Equation S1 gives the sum of mapped construct counts, and Equation S2 gives total input read conservation across the five read classes. In synthetic datasets without sequencing errors, all reads map to constructs in the reference and the two sums are equal. In experimental datasets, they differ by the number of unmapped, invalid, ambiguous, and recombinant read pairs.

S3 Bounded channels and deterministic error reporting

The reader thread groups decompressed records into batches and dispatches them across bounded channels to worker threads. The admission planner can adjust batch sizes while keeping the requested worker count fixed. Workers return local count maps through bounded result channels.

Input and worker errors record the read position where the fault occurred. Error reporting follows fixed precedence rules rather than thread completion order, so error messages are reproducible across multi-threaded runs. Overflow-checked integer addition during reduction ensures that final counts do not depend on thread scheduling.

S4 Format-aware admission planner

The admission planner estimates peak memory across reference loading, parallel counting, and output writing, accounting for single-guide dictionaries, Bloom pre-filter arrays, worker hash tables, bounded channels, recycling queues, and output buffers.

For modeled phase envelopes P_{load}, P_{count}, and P_{output}, the admission condition is:

$$P_{\text{admit}} = \left\lceil \frac{7}{4} \max(P_{\text{load}}, P_{\text{count}}, P_{\text{output}}) \right\rceil \leq B_{\text{memory}} \tag{S3}$$

The admission planner evaluates this inequality using checked integer arithmetic. If the minimum required data structures exceed the configured memory budget, CrisprClamp stops before reading FASTQ input.

S5 Reference-order binary output and atomic publication

CrisprClamp writes an 8-byte unsigned integer count for every construct in reference order, including zero counts for unobserved constructs. A 50-million-construct shard produces an uncompressed count file of 400,000,000 bytes ($8 \times 50{,}000{,}000$), and 20 shards produce 8,000,000,000 bytes in total.

Output file sizes and construct counts are checked against the reference descriptor before completion. Output files are written to a staging directory before atomic publication via directory symlinks so that incomplete files are never exposed.

S6 Optional single-threaded verification profile

In addition to the default multi-threaded mode, CrisprClamp includes an optional single-threaded verification profile (CrisprClamp (Audited)). In this mode, CrisprClamp runs on a single worker thread ($w = 1$), computes streaming file digests, and issues filesystem flush barriers on output files. Because these flush barriers serialize disk writes and reduce speed, CrisprClamp (Audited) is not used in the performance benchmarks.

S7 Complete benchmark observations and shard timing tables

Supplementary Tables S2, S3, and S4 list the benchmark measurements, per-shard workflow times, and summary statistics across all tools and datasets. Values in these tables are rounded to three decimal places, while the CSV tables available from the author on request retain full precision.

Table S1: Classification leaves and read classes for input read pairs. Twelve mutually exclusive classification leaves aggregate into five read classes. Structural FASTQ formatting errors halt execution rather than incrementing a read class.

Classification leaf	Read class	Description
mapped.expected	Mapped	Both guides match a construct in the reference
recombinant.unexpected	Recombinant	Both guides match the guide lists, but the ordered pair is not in the reference
ambiguous.component	Ambiguous	One or both positions match multiple guide sequences
ambiguous.layout	Ambiguous	Multiple extraction windows or anchor alignments match
unmapped.layout	Unmapped	Read sequence does not match the expected flanking structure
unmapped.A_known	Unmapped	Guide A matches, but Guide B does not match any guide in the list

Classification leaf	Read class	Description
unmapped.B_known	Unmapped	Guide B matches, but Guide A does not match any guide in the list
unmapped.neither	Unmapped	Neither position matches a guide in the position-specific lists
invalid.alphabet	Invalid	Read sequence contains non-standard nucleotide characters
invalid.extraction	Invalid	Read is shorter than the configured extraction window
invalid.N	Invalid	Ambiguous N bases occur within the guide extraction window
invalid.quality	Invalid	Phred quality score falls below the configured threshold

Table S2: Synthetic and experimental benchmark measurements. All durations and throughput values are rounded to three decimal places on an Apple M4 Pro workstation (one measured run for each tool on each dataset after warmup). CSV files available from the author on request retain full precision.

Dataset / cell	Tool	Reference constructs	Input pairs	Workflow seconds	Throughput (M pairs/s)
A-100M	CrisprClamp	1,000,000	100,000,000	32.048	3.120
A-100M	DotMatch	1,000,000	100,000,000	73.377	1.363
A-100M	MAGeCK2	1,000,000	100,000,000	145.839	0.686
A-100M	ScreenPro2	1,000,000	100,000,000	452.737	0.221
A-1B	CrisprClamp	1,000,000	1,000,000,000	317.527	3.149
A-1B	DotMatch	1,000,000	1,000,000,000	686.219	1.457
A-1B	MAGeCK2	1,000,000	1,000,000,000	1440.477	0.694
B-50M	CrisprClamp	50,000,000	50,000,000	35.049	1.427
B-50M	DotMatch	50,000,000	50,000,000	192.093	0.260
B-50M	MAGeCK2	50,000,000	50,000,000	243.299	0.206
B-500M	CrisprClamp	50,000,000	500,000,000	141.408	3.536
B-500M	DotMatch	50,000,000	500,000,000	338.579	1.477
B-500M	MAGeCK2	50,000,000	500,000,000	822.011	0.608
GSE116198	CrisprClamp	1,044,484	41,881,752	13.377	3.131
GSE116198	DotMatch	1,044,484	41,881,752	15.425	2.715
GSE116198	MAGeCK2	1,044,484	41,881,752	72.371	0.579
GSE116198	ScreenPro2	1,044,484	41,881,752	161.986	0.259

Table S3: Measured per-shard execution and workflow times. Each shard contains 50,000,000 constructs in the reference and 500,000,000 paired-end reads at 10-fold coverage on an Apple M4 Pro workstation. CrisprClamp and DotMatch completed all 20 shards; MAGeCK2 was measured on shards 0, 1, and 2. CrisprClamp durations are execution times; comparator durations are workflow times. Summary statistics are reported in Supplementary Table S4. Durations are rounded to three decimal places; CSV files available from the author on request retain full precision.

Shard index	CrisprClamp execution seconds	DotMatch workflow seconds	MAGeCK2 workflow seconds
000000	147.737	340.703	816.941
000001	142.656	326.832	836.499
000002	143.070	327.817	823.843
000003	142.900	329.780	Unmeasured
000004	142.914	330.160	Unmeasured
000005	142.773	331.204	Unmeasured
000006	142.480	332.281	Unmeasured
000007	142.713	330.314	Unmeasured
000008	142.766	331.227	Unmeasured
000009	142.737	331.010	Unmeasured
000010	142.710	331.328	Unmeasured
000011	142.583	333.119	Unmeasured
000012	142.382	331.283	Unmeasured
000013	142.830	332.859	Unmeasured
000014	142.813	333.154	Unmeasured
000015	142.578	334.763	Unmeasured

Shard index	CrisprClamp execution seconds	DotMatch workflow seconds	MAGeCK2 workflow seconds
000016	142.960	332.341	Unmeasured
000017	142.853	333.923	Unmeasured
000018	142.750	333.825	Unmeasured
000019	142.606	333.937	Unmeasured

Table S4: Summary statistics for the 10-billion-read workload. All durations are reported in seconds rounded to three decimal places; CSV files available from the author on request retain full precision. CrisprClamp values reflect summed execution time across 20 shards; DotMatch values reflect 20 complete workflow runs; MAGeCK2 values reflect three measured workflow runs and the projected 20-shard total.

Summary metric	CrisprClamp execution s	DotMatch workflow s	MAGeCK2 workflow s
Measured sum	2859.811	6641.860	2477.283
Measured mean	142.991	332.093	825.761
Measured median	142.758	331.804	823.843
Measured minimum	142.382	326.832	816.941
Measured maximum	147.737	340.703	836.499
Projected 20-shard total	Not applicable	Not applicable	16476.863
Projection lower endpoint	Not applicable	Not applicable	16338.820
Projection upper endpoint	Not applicable	Not applicable	16729.982

S8 Development and optimization

We used earlier development builds to examine allocation and batching costs. These measurements describe development builds and do not isolate the contribution of each optimization to the runtime of the final evaluated build. On identical compressed inputs containing 100 million paired reads, with four workers and 512-record batches, the direct-streaming baseline required 52.44 s. Deferring error message construction reduced runtime to 48.89 s, and adding bounded buffer reuse reduced it further to 35.18 s, corresponding to step ratios of 1.1 and 1.4 and a cumulative ratio of 1.5. These tests retained exact scientific validation but do not establish an additive speed decomposition for the final build.

We also evaluated worker and batch geometry during development using three exploratory repetitions per setting. At 100 million paired reads, median complete workflow times were 43.28 s for two workers with 512-record batches and queue capacity two, 35.18 s for four workers with 512-record batches and queue capacity two, and 35.62 s for four workers with 1,024-record batches and queue capacity eight. These observations motivated our default configuration of four workers with 512-record batches. Because the larger-batch setting also changed queue capacity, it does not isolate a pure batch-size effect. The tested settings did not establish a universal optimum, and additional workers remain subject to the startup memory check.

Early development workflows decompressed compressed input to temporary uncompressed files before counting. In the archived development summary, external gzip decompression required 92.84 s, 461.43 s, and 930.15 s at 100 million, 500 million, and 1 billion paired reads, respectively, accounting for 62–63% of those historical workflow times. Direct compressed streaming eliminates this separate disk-staging step while still performing decompression in memory. Because the historical workflows also differed in worker geometry and implementation, their stage times do not measure the speed benefit of streaming decompression in the final build.

S9 Memory measurement and modeled resource limits

We sampled the summed resident memory of the workflow process and its descendants approximately once per second and retained the maximum sampled value. All memory observations in Figure 5

used the same Apple M4 Pro host, input bindings, and process-tree sampling method as the corresponding runtime measurements. Each observation represents one measured run; we did not pool warmup passes, failed invocations, or earlier development-build results, and unmeasured comparator configurations remain unreported. These sampled process-tree values differ from macOS physical-footprint metrics and from the kernel high-water mark of a single native process. For example, the final CrisprClamp native process on the published screen had a separately recorded kernel maximum RSS of 237.9 MiB, whereas Figure 5 consistently uses the sampled process-tree metric (195.2 MiB).

The admission planner evaluates the loading, counting, and output phases using format-specific allocation estimates, including reference membership and filter storage, local and merged count maps, decoder and worker queues, reusable records, parsing buffers, and output serialization. In the final-build log for the published screen, the modeled phase requirements before applying the safety margin were 35,594,312 bytes for loading, 603,886,512 bytes for counting, and 218,008,180 bytes for output. Applying the 7/4 margin yielded 1,056,801,396 bytes, well below the configured 2,147,483,648-byte (2.00 GiB) budget. These values are model outputs for that reference and format rather than measured RSS or phase values from earlier development builds.

At full input geometry, each read stream reserves four queued decoder batches, one producer batch, one consumer-pending batch, and up to 4,096 reusable records. With 512 records per batch, this allocation accounts for 7,168 record slots per stream. Four worker queues hold another 4,096 read pairs, and at most one active batch per worker plus one coordinator batch add 2,560 pairs. Summing these reservations conservatively yields at most 13,824 paired-record slots across the pipeline compartments (with two stream records per pair), rather than holding all reads in memory. Caller records, parser buffers, retained field capacities, and message metadata are accounted for separately. These figures represent capacity bounds rather than observed occupancy or a global allocator ceiling, and the planner can admit smaller capacities under tighter memory budgets.

Table S5: Maximum sampled workflow process-tree RSS. One measured run per observation, sampled approximately once per second. RSS is reported in MiB; exact bytes are available from the author on request. Only matched input and host observations are listed. A-100M and A-1B use 1 million constructs; B-50M and B-500M use 50 million constructs; GSE116198 uses 1,044,484 constructs. Blank or absent observations were not inferred.

Dataset	Tool and version	Paired reads	Sampled RSS (MiB)
GSE116198	CrisprClamp 0.1.0	41,881,752	195.2
GSE116198	DotMatch 0.6.3	41,881,752	119.4
GSE116198	MAGeCK2 0.3.0	41,881,752	2,748.3
GSE116198	ScreenPro2 0.7.0	41,881,752	8,869.0
A-100M	CrisprClamp 0.1.0	100,000,000	425.6
A-1B	CrisprClamp 0.1.0	1,000,000,000	430.0
B-50M	CrisprClamp 0.1.0	50,000,000	4,415.5
B-500M	CrisprClamp 0.1.0	500,000,000	4,414.1
B-50M	DotMatch 0.6.3	50,000,000	44,179.6
B-500M	DotMatch 0.6.3	500,000,000	39,895.1
A-1B	MAGeCK2 0.3.0	1,000,000,000	324.1
B-50M	MAGeCK2 0.3.0	50,000,000	10,758.8
B-500M	MAGeCK2 0.3.0	500,000,000	10,782.4

References

[1] Max A. Horlbeck, Albert Xu, Min Wang, et al. Mapping the Genetic Landscape of Human Cells. *Cell*, 174:953–967.e22, 2018.

[2] John Paul Shen, Dongxin Zhao, Roman Sasik, et al. Combinatorial CRISPR–Cas9 screens for de novo mapping of genetic interactions. *Nature Methods*, 14:573–576, 2017.

[3] Phoebe C. R. Parrish, James D. Thomas, Austin M. Gabel, et al. Discovery of synthetic lethal and tumor suppressor paralog pairs in the human genome. *Cell Reports*, 36:109597, 2021.

[4] Nicola A. Thompson, Marco Ranzani, Louise van der Weyden, et al. Combinatorial CRISPR screen identifies fitness effects of gene paralogues. *Nature Communications*, 12:1302, 2021.

[5] John G. Doench, Nicolo Fusi, Meagan Sullender, et al. Optimized sgRNA design to maximize activity and minimize off-target effects of CRISPR-Cas9. *Nature Biotechnology*, 34:184–191, 2016.

[6] Fadi J. Najm, Christine Strand, Katherine F. Donovan, et al. Orthologous CRISPR–Cas9 enzymes for combinatorial genetic screens. *Nature Biotechnology*, 36:179–189, 2018.

[7] Peter C. DeWeirdt, Kendall R. Sanson, Annabel K. Sangree, et al. Optimization of AsCas12a for combinatorial genetic screens in human cells. *Nature Biotechnology*, 39:94–104, 2021.

[8] Michael Boettcher, Ruilin Tian, James A. Blau, et al. Dual gene activation and knockout screen reveals directional dependencies in genetic networks. *Nature Biotechnology*, 36:170–178, 2018.

[9] Joseph M. Replogle, Thomas M. Norman, Albert Xu, et al. Combinatorial single-cell CRISPR screens by direct guide RNA capture and targeted sequencing. *Nature Biotechnology*, 38:954–961, 2020.

[10] Joseph M. Replogle, Reuben A. Saunders, Angela N. Pogson, et al. Mapping information-rich genotype-phenotype landscapes with genome-scale Perturb-seq. *Cell*, 185:2559–2575.e28, 2022.

[11] D. O'Toole. DotMatch: deterministic known-target short-DNA assignment for sequencing workflows (version 0.6.3) [software]. Zenodo, 2026. doi: 10.5281/zenodo.22998373.

[12] Wei Li, Han Xu, Tengfei Xiao, et al. MAGeCK enables robust identification of essential genes from genome-scale CRISPR/Cas9 knockout screens. *Genome Biology*, 15:554, 2014.

[13] W. Li et al. MAGeCK2: Model-based Analysis of CRISPR screens (version 0.3.0) [software], 2026. Available from: `https://github.com/davidliwei/mageck2`.

[14] Abolfazl Arab et al. ScreenPro2 (version 0.7.0) [software]. Arc Institute; Zenodo, 2026. doi: 10.5281/zenodo.18884467.

[15] T. A. O'Loughlin, A. Arab, and L. A. Gilbert. Chemogenomic maps reveal a PRDX1-dependent iron-damage axis in the DNA damage response. *Nature Chemical Biology*, 2026. doi: 10.1038/s41589-026-02312-z.

[16] Binbin Wang, Mei Wang, Wubing Zhang, et al. Integrative analysis of pooled CRISPR genetic screens using MAGeCKFlute. *Nature Protocols*, 14:756–780, 2019.

[17] Teemu Kivioja, Anna Vähärautio, Kasper Karlsson, et al. Counting absolute numbers of molecules using unique molecular identifiers. *Nature Methods*, 9:72–74, 2012.

[18] Georg Michlits, Maria Hubmann, Szu-Hsien Wu, et al. CRISPR-UMI: single-cell lineage tracing of pooled CRISPR-Cas9 screens. *Nature Methods*, 14:1191–1197, 2017.

[19] Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13:422–426, 1970.

[20] Adam Kirsch and Michael Mitzenmacher. Less hashing, same performance: Building a better Bloom filter. In *Algorithms – ESA 2006*, volume 4168, pages 456–467, 2006.

[21] Peter J. A. Cock, Christopher J. Fields, Naohisa Goto, et al. The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants. *Nucleic Acids Research*, 38:1767–1771, 2010.